

SIH 2026

DEVELOPER 2 — BLOCKCHAIN MICROSERVICE PRD

Hyperledger Fabric • Gateway • Chaincode • Trusted Ledger

Architecture context: *This PRD defines one independently owned microservice in the agreed three-service architecture. The services are Data & Storage, Blockchain, and Application/Backend. They communicate through explicit internal contracts.*

40-hour Base Model: Build the smallest complete vertical slice first. EPCIS/event streaming, IoT, AI/ML and advanced privacy remain future modules.

1. Mission

The Blockchain Service is the trusted-ledger boundary. It owns Hyperledger Fabric network configuration, MSP identities, channel, one chaincode package and Fabric Gateway integration. It exposes a stable internal API to the Application Service.

2. Ownership

Owned by Developer 2	Explicitly not owned
Fabric network, peers/orderer/channel	PostgreSQL/Redis/IPFS
MSP identities and Gateway	FastAPI public API
TraceabilityContract	Rich operational read model
IncidentContract	Large evidence files
AuditContract	Frontend
Transaction/query API	Risk analytics implementation

3. Architecture

Application Service → internal REST API → Blockchain Service → Fabric Gateway → Hyperledger Fabric

4. Network

1. Provide a reproducible permissioned development network.
2. Configure required organizations/MSPs and identities.
3. Configure a Base Model channel.
4. Provide peers and ordering service sufficient for the demonstration.
5. Provide Gateway connectivity.
6. Document network start/stop/deploy commands.

5. Chaincode package

One chaincode package with three logical contracts. Do not create a separate chaincode package for every feature.

Contract	Responsibility
TraceabilityContract	Product, batch, unit, lifecycle, custody, receipt, transfer, transformation and trusted lineage references.
IncidentContract	Incident status, accountability/escalation commitments, block and recall actions.
AuditContract	Scan and inner-credential verification audit events; audit-only interactions.

6. Trusted assets

Asset	Minimum trusted fields
Product	ID, type/name, creator org, timestamp, status/reference
Batch	ID, product ID, state, custodian/org, parent/reference, timestamps
Unit	ID, batch/product IDs, state, custodian/reference, timestamps
Incident	ID, entity, category/status, reporter/reference, timestamps, evidence CID where required
Recall	ID, affected scope, authority, status, timestamp
Audit Event	ID, type, entity, actor/org, timestamp, transaction context

7. TraceabilityContract

Function	Requirement
registerProduct	Create trusted product; reject duplicates
registerBatch	Create trusted batch linked to product
validateBatch	Permit only valid validation transition

Function	Requirement
receiveBatch	Record explicit custody receipt
transferBatch	Record explicit custody transfer
createTransformation	Record parent/child transformation
createUnit	Create traceable unit from batch

8. Critical rules

1. Chaincode independently verifies actor/organization authority.
2. Chaincode rejects invalid lifecycle transitions.
3. Current-custodian checks apply to custody operations.
4. Scanning never changes custody.
5. Transformation verifies input state and actor authority.
6. Duplicate product/batch/unit identities are rejected.

SCAN → AuditContract.recordScan()

RECEIVE → TraceabilityContract.receiveBatch()

TRANSFER → TraceabilityContract.transferBatch()

9. IncidentContract

1. Create incident reference linked to product/batch/unit.
2. Update incident status/investigation.
3. Record authorized escalation.
4. Block affected batch/unit scope.
5. Create trusted recall action.
6. Close incident after authorized corrective action.

10. AuditContract

Scan and verification are auditable interactions. They must not mutate custody or lifecycle state.

Function	Purpose
recordScan	Record actor/context/entity/time/credential reference and audit result.
recordVerification	Record inner-credential verification interaction/result.

11. Evidence

Evidence file → IPFS CID → Data Service metadata → relevant Fabric CID/reference

Fabric stores compact references/integrity information, not large evidence files.

12. Internal API

Method	Endpoint	Purpose
POST	/internal/transactions/products	Register product
POST	/internal/transactions/batches	Register batch
POST	/internal/transactions/batches/{id}/validate	Validate batch
POST	/internal/transactions/receive	Receive
POST	/internal/transactions/transfer	Transfer
POST	/internal/transactions/transform	Transform
POST	/internal/transactions/units	Create unit
POST	/internal/audit/scan	Record scan
POST	/internal/audit/verify	Record verification
POST	/internal/incidents	Incident operation

Method	Endpoint	Purpose
POST	/internal/blocks	Block scope
POST	/internal/recalls	Recall
GET	/internal/ledger/{entity_id}	Trusted state/reference

13. Transaction result

```
{ transaction_id, status, entity_id, event_type, committed_at, resulting_state, ledger_reference }
```

14. Security

1. Use Fabric MSP identity as trusted actor identity.
2. Do not trust role/org fields only from request payloads.
3. Protect private keys/certificates outside source control.
4. Verify authorization and state in chaincode.
5. Keep sensitive consumer data minimal on-chain.

15. Testing

1. Chaincode unit tests.
2. Authorization and invalid-state tests.
3. Custody and transformation tests.
4. Audit tests proving scan does not change custody.
5. Incident/block/recall tests.
6. Gateway integration tests.
7. Product → Batch → Validate end-to-end test.

16. Phase 1 deliverables

1. Runnable Fabric network and identities.
2. Channel.
3. One chaincode package with three contracts.
4. Product/batch assets.
5. registerProduct, registerBatch and validateBatch.
6. Gateway service and internal API.
7. Deployment/invocation documentation.
8. Automated tests.

17. Definition of Done

The Blockchain Service runs independently, performs trusted Product → Batch → Validate transactions, rejects invalid/unauthorized transitions, returns transaction IDs, and lets Developer 3 use Fabric without knowing Fabric network internals.